Chapter 9

Relational Databases

Why this chapter matters: in almost every system the database is the hardest tier to scale. Application servers are stateless and cloneable (Chapters 5, 7, 8); the database holds the **state**, must stay correct, and is usually the first real bottleneck. Understanding how a relational database stores data, guarantees correctness, and can be scaled — **replication, federation, sharding, denormalization and tuning** — is core to every senior design discussion.

⚡ **QUICK RECALL — Chapter map**

- 9.1 Relational model → 9.2 ACID, transactions & isolation → 9.3 Storage & indexes → 9.4 SQL tuning
- 9.5 Scaling reads: replication (leader–follower, multi-leader) → 9.6 Scaling writes & size: federation & sharding → 9.7 Denormalization & materialized views
- 9.8 Connection pooling → 9.9 Backup, HA & zero-downtime migrations → 9.10 Scaling ladder & when to leave relational
- 9.11 Worked examples → 9.12 Practice, Q&A, cheat sheet

9.1 The relational model

A **relational database management system (RDBMS)** such as PostgreSQL, MySQL (InnoDB), SQL Server or Oracle is a collection of data items **organised in tables** (rows and columns) with a fixed schema, queried with SQL. Relationships between tables are expressed with keys and combined at query time with **joins**.

Concept	Meaning	Example
Primary key (PK)	Uniquely identifies a row; automatically indexed	`users.id`
Foreign key (FK)	Column referencing another table's PK; the DB enforces **referential integrity**	`orders.user_id → users.id`
Constraints	UNIQUE, NOT NULL, CHECK — the database rejects invalid data	`email UNIQUE NOT NULL`
Join	Combine rows from tables on matching keys (INNER, LEFT, …)	`orders JOIN users ON ...`
Normalisation	Organise data to remove redundancy: **1NF** atomic values; **2NF** no partial dependency on part of a composite key; **3NF** no transitive dependency (non-key columns depend only on the key)	Store the customer's name once in `users`, not in every order

★ **MUST REMEMBER — When a relational database is the right default**

- **Structured data**, strict schema, **relational data**, need for **complex joins** and ad-hoc queries, **transactions** (ACID), mature tooling and community, clear patterns for scaling, and **lookups by index are very fast** (Chapter 10 compares with NoSQL).
- Normalise to keep writes simple and data consistent; **denormalise selectively** later when reads need it (9.7).

9.2 ACID, transactions & isolation

ACID is the set of properties of relational database **transactions** (a group of operations treated as one unit):

	Property	Meaning	How it is achieved
A	**Atomicity**	Each transaction is **all or nothing**: if any step fails, everything is rolled back	Undo log / write-ahead log (WAL); rollback

	Property	Meaning	How it is achieved
C	**Consistency**	Any transaction brings the database **from one valid state to another** (constraints, keys, triggers always hold)	Constraints + application invariants (the app defines 'valid'; not the same C as in CAP, Chapter 5)
I	**Isolation**	Executing transactions **concurrently has the same results as if they were executed serially**	Locks and/or **MVCC** (multi-version concurrency control) snapshots; levels below
D	**Durability**	Once a transaction has been **committed, it will remain so** — even after crash or power loss	WAL flushed with `fsync` at commit; replication (Chapter 2)

```
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1 AND balance >= 100;  -- check rows affected = 1
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;            -- both changes become durable together;  ROLLBACK on any error -> neither happens
```

Isolation levels and anomalies

Perfect isolation (serializable) is expensive, so databases offer weaker levels that allow certain anomalies:

Level	Dirty read	Non-repeatable read	Phantom read	Notes
Read uncommitted	possible	possible	possible	Rarely used
Read committed	no	possible	possible	**Default in PostgreSQL, Oracle, SQL Server**; each statement sees only committed data
Repeatable read	no	no	depends on engine	**Default in MySQL InnoDB**; transaction sees one consistent snapshot (PostgreSQL's snapshot also blocks phantoms but allows write skew)
Serializable	no	no	no	Strongest; may abort transactions → **application must retry**; lower throughput

Dirty read: see another transaction's uncommitted data. **Non-repeatable read**: the same row read twice returns different values. **Phantom**: the same query returns new rows. **Lost update**: two transactions read, then both write — one overwrites the other. **Write skew**: two transactions each check a condition on overlapping data and write different rows, jointly breaking an invariant (two doctors both go off-call).

Handling concurrency in application code

Technique	How	Use when
Atomic update	`UPDATE stock SET qty = qty - 1 WHERE id=42 AND qty > 0` — check/modify in one statement	Counters, inventory — the first choice
Pessimistic lock	`SELECT ... FOR UPDATE` locks the rows until commit	High contention, short transactions
Optimistic lock	Version column: `UPDATE ... SET v = v+1 WHERE id=42 AND v=7`; 0 rows affected ⇒ conflict, retry	Low contention, long think-time (user edits)
Serializable + retry	Let the DB detect conflicts; retry on serialization failure	Complex invariants (write skew)

◆ **SENIOR INSIGHT** — **Locks, deadlocks and MVCC in practice**

- **MVCC** (PostgreSQL, MySQL InnoDB, Oracle) keeps multiple row versions so **readers don't block writers and writers don't block readers**; old versions are cleaned up later (VACUUM in PostgreSQL — bloat and long-running transactions are classic production problems).
- **Deadlock**: two transactions each hold a lock the other needs (Chapter 1's four conditions). The DB detects it and aborts one — your code must **retry**. Reduce by locking rows in a consistent order and keeping transactions short.
- **Never hold a transaction open across a network call** (HTTP, queue) — it holds locks and a connection for an unbounded time.
- Across services you can't have one ACID transaction — use sagas and the outbox pattern (Chapter 8).

9.3 How data is stored & why indexes matter

- Data lives in fixed-size **pages** (8 KB PostgreSQL, 16 KB InnoDB). Hot pages are cached in RAM by the **buffer pool**; changes are first appended to the **WAL/redo log** (sequential I/O, Chapter 2) and written to data files later.
- Without an index the database does a **full table scan**: read every row (O(n)). An index lets it jump straight to matching rows.

Indexes are usually represented as a **self-balancing B-tree** that keeps data sorted and allows **searches, sequential access, insertions and deletions in logarithmic time**.

```
                  [ 40 | 80 ]                      <- root page (always cached)
                 /      |      \
        [10|20|30]   [50|60|70]   [90|100|110]      <- internal pages
          /  |  \
     leaf pages: sorted keys + row pointers (or the rows themselves in a clustered index)

Lookup cost = tree height = log base fan-out of N.   Fan-out ~200-500  ->  100M rows = ~4 levels
=> about 4 page reads (top 2-3 levels sit in cache)  vs  reading all 100M rows for a full scan.
```

Index concept	What to know
Clustered vs secondary	InnoDB stores the table *as* a B-tree on the primary key (clustered). Secondary indexes store the PK, so a lookup may need a second probe. PostgreSQL tables are heaps; all indexes point to row locations
Composite index & leftmost prefix	`INDEX(user_id, created_at)` serves `WHERE user_id=?` and `WHERE user_id=? ORDER BY created_at`, but **not** a filter on created_at alone. Order columns: equality first, then range/sort
Covering index	Index contains every column the query needs → answered from the index alone (no table access)
Selectivity / cardinality	Indexes pay off when they narrow results a lot. Indexing a boolean (50/50) rarely helps; the planner may ignore it
Other kinds	Hash (equality only), partial/filtered (index only `WHERE active`), full-text, GIN/GiST (JSON, arrays, geo), unique indexes enforce constraints
Costs	Placing an index can keep data in memory, **requiring more space**; **writes could also be slower since the index also needs to be updated**; too many indexes hurt inserts/updates

✖ **COMMON MISTAKE** — **Why your index isn't used**

• Function/expression on the column (`WHERE LOWER(email)=?` without an expression index) • leading wildcard (`LIKE '%abc'`) • type mismatch (string vs integer) • low selectivity • stale statistics • `OR` across different columns • implicit casts. **Always check with** `EXPLAIN (ANALYZE)` instead of guessing.

9.4 SQL tuning

SQL tuning is a broad topic and many books have been written as reference. The rule: **benchmark and profile first** to simulate and uncover bottlenecks, then change one thing at a time.

Step	Tools
Benchmark — simulate high-load situations	ab (ApacheBench), wrk, k6, `pgbench`, sysbench
Profile — find the slow queries	**Slow query log** (MySQL), `pg_stat_statements` (PostgreSQL), APM/tracing, `EXPLAIN ANALYZE`, `SHOW PROCESSLIST`/`pg_stat_activity`

Tighten up the schema

Guideline (from your notes)	Notes & corrections
MySQL dumps to disk in contiguous blocks for fast access	Compact, fixed-width rows and clustered-key order improve scan speed and cache use
Use `CHAR` instead of `VARCHAR` for **fixed-length** fields	CHAR allows fast random access; with VARCHAR you must find the end of a string before the next one. Gain is small in modern engines — use CHAR for truly fixed codes (country, currency)
Use `TEXT` for large blocks of text such as blog posts	TEXT supports full-text search; stored out-of-line with a pointer. Keep large text out of hot, frequently-read tables
Use `INT` for larger numbers up to 2^32 or 4 billion	**Correction:** signed INT max is $2^{31}-1 \approx 2.1$ billion; **unsigned** reaches 4.29 billion. Use `BIGINT` for ids that may exceed this
Use `DECIMAL` for currency to avoid floating-point errors	Correct — or store integer minor units (cents) with the currency code
Avoid storing large `BLOB`s; store the location of the object instead	Correct: put files in object storage + CDN (Chapters 2, 6), keep the URL/key in the row
`VARCHAR(255)` is the largest count in an 8-bit length byte	Historical MySQL detail; choose lengths from real requirements
Set `NOT NULL` where applicable to improve search performance	**Nuance:** main benefit is correctness and slightly simpler indexes/storage; performance gain is usually small

Query & access patterns

- **Use good indices** on columns you are querying with `SELECT`, `GROUP BY`, `ORDER BY` and `JOIN` (9.3). When loading large amounts of data, it might be faster to **disable indices, load the data, then rebuild the indices**.
- **Avoid expensive joins**: denormalise where performance demands it (9.7). Select only needed columns, never `SELECT *` in hot paths.
- **Avoid the N+1 query problem** (ORM lazy loading: 1 query for 100 orders + 100 queries for each customer) → join/`IN` batch/eager loading.
- **Paginate with keyset** (`WHERE id > :last ORDER BY id LIMIT 20`), not deep OFFSET (Chapter 4). **Batch** writes (multi-row INSERT, COPY) — one round trip instead of thousands.
- **Partition tables**: break up a table (by date range, list or hash) — e.g., by putting hot spots in a separate table to help keep it in memory; old partitions can be dropped or archived instantly. *Partitioning is within one database; sharding (9.6) spans servers.*
- **Tune the query cache**: in some cases the query cache can lead to performance issues (global lock, invalidation on every write). **MySQL 8.0 removed it**; use an application cache (Redis, Chapter 11) instead.
- **Keep transactions short**, use the lowest sufficient isolation level, and set statement/lock timeouts so one bad query can't hold the system hostage.

9.5 Scaling reads: replication

There are many techniques to scale a relational database: **master–slave replication, master–master replication, federation, sharding, denormalization and SQL tuning**. Start with replication — most applications are read-heavy (reads often outnumber writes 100:1 or even 1000:1). Modern terminology: **primary/replica** or **leader/follower**.

Leader–follower (master–slave) replication

```
Client --- reads + writes --->  [ LEADER ] --- async / semi-sync replication ---> [ Follower 1 ]
                                                                                  [ Follower 2 ]
Client --- reads only ------------------------------------------------------->  (followers can
                                                                                  feed more followers:
If the leader goes offline: continue read-only until a follower is                a tree)
PROMOTED or a new leader is provisioned.
```

- The **leader serves reads and writes**, replicating writes to one or more followers, which serve **only reads**. Followers can also replicate to additional followers in a **tree-like fashion**.
- Replication ships the **WAL / binlog** (physical or logical). Sync vs async vs semi-sync trade-offs: Chapter 5 (RPO and latency).
- **Read/write splitting**: the application (or a proxy such as ProxySQL, pgpool, RDS Proxy) sends writes to the leader and reads to followers. Reads needing fresh data → leader (read-your-writes, Chapter 5).

Multi-leader (master–master) replication

Both masters serve reads and writes and coordinate with each other on writes. If either master goes down, the system can continue to operate with both reads and writes. Typical uses: multi-region writes (low local latency), offline-capable clients, active–active HA.

	Leader-follower	Multi-leader
Writes go to	One leader	Any leader
Failure of leader	Read-only until promotion; **additional logic is needed to promote a follower to leader** (Patroni, RDS Multi-AZ automate it)	Other leader keeps serving reads and writes
Consistency	Simple; followers lag behind	**Conflicts** (same row edited in two places) need resolution: last-write-wins, merge rules, CRDTs, app logic
Other drawbacks	Single write bottleneck	You need a **load balancer or application logic to determine where to write**; most multi-leader systems are either loosely consistent (violating ACID) or have increased write latency due to synchronisation; conflict resolution matters more as write nodes and latency increase; auto-increment ids collide (use offsets or UUID/Snowflake ids)

Disadvantages of replication (all types)

- **Potential for data loss** if the master fails before newly written data can be replicated to other nodes (asynchronous).
- **Writes are replayed on the read replicas.** If there are a lot of writes, the read replicas can get bogged down replaying writes and can't do as many reads.
- **The more read replicas, the more you have to replicate**, which leads to greater **replication lag**.
- On some systems, writing to the master can spawn multiple threads to write in parallel, whereas read replicas only support writing **sequentially with a single thread** — so replicas can fall permanently behind under heavy write load.

- Replication adds more hardware and additional complexity.

> ◆ **SENIOR INSIGHT** — **Replication lag in the real world**
> - Symptoms: user saves, reloads, and doesn't see the change; reports and jobs read stale data; failover loses recent writes. Monitor **lag in seconds and bytes**, alert on it, and route lag-sensitive reads to the leader or wait for a log position (version token, Chapter 5).
> - Replication scales **reads and availability, not writes**: all writes still hit one leader. When write volume or data size outgrows one machine, you need federation or sharding (9.6).
> - A replica is **not a backup** — an accidental `DROP TABLE` replicates instantly (Chapter 2). Use point-in-time recovery (9.9).

9.6 Scaling writes & data size: federation and sharding

Federation (**functional partitioning**) splits up databases **by function**. Instead of a single, monolithic database, you could have **three databases: forums, users and products**, each with its own leader and followers.

```
Before:  one big database (users + forums + products)
After :  Forums DB  | Users DB  | Products DB      (each with its own leader + followers)
```

Federation advantages	Federation disadvantages
Less read and write traffic to each database, therefore less replication lag **Smaller databases** → more data fits in memory → more cache hits due to improved cache locality With **no single central master serializing writes** you can write in parallel, increasing throughput	**Not effective if your schema requires huge functions or tables** (one function can still outgrow a server) You'll need to **update your application logic** to determine which database to read and write **Joining data from two databases is more complex** (server link / application-side joins) Adds more hardware and additional complexity

Sharding distributes data across different databases such that **each database can only manage a subset of the data**. Taking a users database as an example, as the number of users increases, more shards are added to the cluster.

```
Application --- User[Adam] ---> [ Shard router: proxy / load balancer / client library ]
                                      |           |           |              |
                                  Shard A-C   Shard D-F   Shard G-I  ...  Shard X-Z
                                  (each shard = a leader + followers)
```

Similar to the advantages of federation, sharding results in **less read and write traffic, less replication and more cache hits**. **Index size is also reduced**, which generally improves performance with faster queries. If one shard goes down, the other shards are still operational, although you'll want to add some form of replication to avoid data loss. Like federation, there is no single central master serializing writes, allowing you to write in parallel with increased throughput.

Sharding strategies & the shard key

Strategy	How	Pros	Cons
Range	Key ranges per shard (ids 1–10M, 10M–20M; or dates)	Simple; efficient range scans	**Hot spots**: newest range takes all writes; uneven sizes
Hash	`shard = hash(key) mod N` (or consistent hashing)	**Even distribution**, no hot ranges	Range queries scatter across shards; **mod N reshuffles almost every key when N changes**
Directory / lookup	A mapping service stores key → shard	Flexible; move individual tenants	Lookup is an extra hop and a SPOF/bottleneck (cache it)
Geographic	Region/country → shard	Low latency, data residency	Uneven load per region

◆ **SENIOR INSIGHT** — **Choosing a good shard key — the most important decision**

- Properties: **high cardinality**, **even distribution** (no hot keys), **matches the dominant query pattern** (so most queries touch one shard), and **stable** (rarely changes). For a SaaS app, `tenant_id`; for a social app, `user_id`.
- Common ways to shard a table of users are the **last-name initial** or **geographic location** (your notes) — fine for explaining the idea, but in practice names and regions are heavily skewed (many surnames start with the same letters; one country dominates). Prefer a **hash of an id**.
- **Celebrity / hot key** problem: one shard receives disproportionate load (power users). Mitigate with caching, splitting hot keys, dedicated shards for whales.

Costs of sharding & how teams cope

Disadvantage	Details	Mitigation
Application complexity	You'll need to update your application logic to work with shards, which could result in complex SQL queries	Routing layer/proxy (Vitess, Citus, ProxySQL), ORM sharding support, data-access library
Lopsided distribution & rebalancing	Data distribution can become lopsided (a set of power users on a shard); **rebalancing adds additional complexity**	**Consistent hashing** can reduce the amount of transferred data (Chapter 14); or **many logical shards** (e.g., 1,024) mapped to few physical servers — move whole logical shards later
Cross-shard joins & transactions	Joining data from multiple shards is more complex; no single ACID transaction	Co-locate related data on the same shard key; denormalise; sagas (Chapter 8); avoid cross-shard queries on hot paths
Global uniqueness	Auto-increment ids collide across shards	Snowflake-style ids (time + shard + sequence), UUIDv7, ID service/ticket server
Global secondary indexes	A query by email when sharded by user_id must hit every shard	Separate lookup table/index service keyed by email → user_id
More hardware, more ops	Sharding adds more hardware and additional complexity	Automation, managed services, NewSQL (Spanner, CockroachDB, Vitess)

Partitioning vs federation vs sharding

	Table partitioning	Federation	Sharding
Splits	One table into parts, **inside one DB server**	**Different tables/functions** onto different DBs	**Rows of the same table** across DBs/servers
Main gain	Faster queries (partition pruning), easy archiving, smaller indexes	Isolation of domains, more write throughput per domain	Horizontal scaling of writes and data size
Query impact	Transparent to the app	App must pick the DB; cross-DB joins hard	App/proxy must pick the shard; cross-shard queries hard

9.7 Denormalization & materialized views

Denormalization attempts to **improve read performance at the expense of some write performance**. Redundant copies of the data are written in multiple tables to **avoid expensive joins**. Once data becomes distributed with federation and sharding, managing joins across data centres further increases complexity — denormalization might circumvent the need for such complex joins.

```
Normalised (3 joins per page view)
orders(id, user_id, ...)
users(id, name, city_id)
cities(id, name)
order_items(order_id, product_id, ...)
```

```
Denormalised read model (1 lookup)
order_view(order_id, user_name, user_city,
           item_titles[], total, status)
-> updated whenever user/order/item data changes
```

How to keep copies in sync	Notes
Materialized views	Some RDBMS such as **PostgreSQL and Oracle support materialized views**, which handle the work of storing redundant information and keeping redundant copies consistent (refresh on schedule/commit; incremental in some engines)
Triggers / DB constraints	Constraints can help redundant copies stay in sync, which increases complexity of the database design
Application dual writes	Simple but risky (partial failure → drift); use transactions or the outbox pattern (Chapter 8)
CDC (change data capture) / events	Stream changes (Debezium + Kafka) into read models, search indexes, caches — eventual consistency
Caches / precomputed counters	Counters (`likes_count`), aggregates, per-user timelines (Chapter 11)

★ **MUST REMEMBER** — **Disadvantages of denormalization**

- **Data is duplicated** (more storage, risk of inconsistency). Constraints/sync logic increase the complexity of the database design. **A denormalized database under heavy write load might perform worse than its normalized counterpart.**
- Use it where **reads dominate writes** (100:1–1000:1) and a join would be a significant cost spent on disk operations; keep the normalised tables as the source of truth.

9.8 Connection management & pooling

A database connection is expensive: PostgreSQL uses a process (several MB) per connection; MySQL a thread; each does TCP + auth (+ TLS) setup (Chapter 3). Too many connections cause memory pressure and context-switch overhead (Chapter 1) — throughput **drops** beyond a sweet spot (often tens to a few hundred active connections per server).

```
const { Pool } = require('pg');
const pool = new Pool({
  max: 20,                        // max connections THIS process may open
  idleTimeoutMillis: 30_000,      // close idle connections after 30 s
  connectionTimeoutMillis: 2_000, // fail fast if the pool is exhausted
});
const { rows } = await pool.query('SELECT * FROM users WHERE id = $1', [id]);  // parameterised
```

- **Total connections = instances × pool size**. 40 app instances × 20 = **800** connections — more than most databases want. This is the “downstream servers must handle more simultaneous connections as upstream servers scale out” problem from Chapter 7.
- **External poolers** (PgBouncer, RDS Proxy, ProxySQL) multiplex thousands of client connections onto a small number of real ones (transaction-level pooling). Caveat: session state (prepared statements, SET, advisory locks) may not survive transaction pooling.
- **Sizing** with Little’s law (Chapter 2): connections needed ≈ queries/s × average query time. 2,000 queries/s × 10 ms = **20** busy connections — a small pool is enough if queries are fast; long queries need big pools *or* fixing.
- Always set **timeouts** (acquire, query, idle-in-transaction) so a slow query cannot starve the pool and take the service down.

9.9 Backup, high availability & zero-downtime migrations

Topic	Practice
Backups	Periodic **base backup/snapshot** + continuous **WAL/binlog archiving** = **point-in-time recovery (PITR)**: restore to "5 minutes before the bad deploy". Store off-site, encrypted, versioned (3-2-1, Chapter 2). **Test restores regularly** — an untested backup is a hope, not a backup
High availability	Synchronous or semi-sync standby in another AZ with **automatic failover** (Patroni, Orchestrator, RDS/Aurora Multi-AZ); fencing to avoid split brain (Chapter 5); clients reconnect with retries and DNS/endpoint that follows the leader
RPO / RTO	Choose per business need: sync replication → RPO≈0; async/backups → minutes-hours. Cross-region replicas for disaster recovery
Capacity & storage	Watch disk, IOPS, connections, replication lag, cache hit ratio, long transactions, bloat; plan growth (storage = rows × size × indexes overhead × replicas)
Security	Least-privilege DB roles, network isolation (private subnets), TLS, encryption at rest, secrets rotation, parameterised queries (Chapter 13)

Schema changes without downtime — the expand / contract pattern

1. **Expand**: add the new column/table as **nullable** (cheap, backward compatible); avoid operations that lock or rewrite the whole table (use online DDL tools: gh-ost, pt-online-schema-change, `CREATE INDEX CONCURRENTLY`).
2. **Deploy code that writes both** old and new fields (dual-write) and reads the old.
3. **Backfill** existing rows in small batches (rate-limited, off-peak) so replicas don't lag.
4. **Switch reads** to the new field; verify; keep rollback ready.
5. **Contract**: stop writing the old field, then drop it in a later release.

> ✖ **COMMON MISTAKE — Dangerous migrations**
>
> • Adding a column with a default or an index on a huge table in one blocking statement; renaming/dropping a column still used by the running version; long backfills in one transaction. Rule: **every migration must be compatible with both the old and new application version**, because during a rolling deploy both run at once.

9.10 The relational scaling ladder & when to look beyond

Climb only as far as the numbers force you. Each rung adds complexity.

#	Step	Fixes	New cost / limit
1	**Tune**: indexes, queries, schema, connection pool, config	Most 'slow DB' problems	None — always do first
2	**Vertical scale** (more RAM/CPU/NVMe)	Quick relief; more data fits in the buffer pool	Ceiling; SPOF; cost
3	**Cache** hot reads (Redis, Chapter 11)	Removes most read load	Invalidation, staleness
4	**Read replicas** + read/write splitting	Read scalability, HA	Replication lag; writes unchanged
5	**Partition** large tables; archive old data	Faster queries, cheap deletes	Operational care
6	**Federation** (split by function)	Write scaling per domain; isolation	Cross-DB joins
7	**Sharding**	Write & data-size scaling	Cross-shard queries/transactions; rebalancing; big engineering cost

#	Step	Fixes	New cost / limit
8	**NewSQL** (Spanner, CockroachDB, Vitess, Aurora) or **NoSQL** (Chapter 10)	Built-in distribution	Different guarantees, latency, learning curve

◆ **SENIOR INSIGHT** — **How far can one relational node go?**

- A well-tuned PostgreSQL/MySQL instance on modern hardware commonly serves **thousands to tens of thousands of simple queries per second** and **terabytes of data**. Many companies reach hundreds of millions of users before sharding. **Don't shard on day one** — estimate (Chapter 2) first.
- Consider non-relational stores when: data is **semi-structured or schema-flexible**, joins are unnecessary, you must store **many TB–PB** or need **very high write throughput/IOPS**, or the access pattern is simply key-value, wide-column or graph (Chapter 10).

9.11 Worked examples

Example A — Index vs full scan (100 M-row table, ~100 bytes/row, B-tree fan-out ≈ 200)

```
Index height  = log_200(100,000,000) = 8 / 2.30 = 3.5   -> 4 levels  -> ~4 page reads (top levels cached)
Full scan     = 100 M rows x 100 B = 10 GB; at ~1 GB/s sequential SSD read -> ~10 s (evicts cache)
Point lookup with the index ~ well under 1 ms from cache / ~1 ms from SSD:  >1000x faster.
```

Example B — How many shards? (500 M users × 2 KB = 1 TB; peak 20,000 writes/s; one node sustains ~5,000 writes/s and 500 GB comfortably)

Constraint	Calculation	Minimum shards
Write throughput	20,000 ÷ 5,000	**4**
Data size (1 TB now; ×3 in 3 years = 3 TB)	3 TB ÷ 0.5 TB	**6**
Headroom, rebalancing, future growth	Round up to a power of two	**16 logical shards**
With 1 leader + 2 followers per shard	16 × 3	**48 database nodes** (or start with 4 physical nodes hosting 16 logical shards and split later)

Example C — Connection arithmetic

40 API instances × pool 20 = 800 connections; the database allows ~200. Fix: PgBouncer in transaction mode (800 client connections → 40 server connections), and reduce per-instance pool to 5–10. Check with Little's law: 4,000 queries/s × 5 ms = 20 busy connections needed — plenty of headroom.

Example D — Replication lag bug

A user updates their profile (write → leader) and the redirect immediately reads from a follower 800 ms behind: the old name appears. Fixes in order of simplicity: read the user's own data from the leader for 5–10 s after a write; return the updated object in the write response; or send a version token (LSN) and let the follower wait until it has caught up.

9.12 Practice questions (try before looking)

P1. Two requests both try to buy the last item (stock = 1). How do you prevent overselling?

Answer: Single atomic statement: `UPDATE items SET stock = stock - 1 WHERE id=? AND stock > 0` and check affected rows = 1; or `SELECT ... FOR UPDATE` inside a transaction; or optimistic version check with retry.

P2. Your read-heavy app has 90% CPU on the primary. List what to try, in order.

Answer: Profile slow queries/indexes → connection pooling → cache hot reads → add read replicas with read/write splitting → partition/archive → only then federation/sharding.

P3. You shard users by `hash(user_id) mod 4` and now need 5 shards. Problem?

Answer: Mod N remaps most keys (~80% move). Use consistent hashing or many fixed logical shards mapped to physical nodes so only a few logical shards move.

P4. Why is an index on `(created_at, user_id)` bad for `WHERE user_id = ?`?

Answer: Leftmost-prefix rule: the index is ordered by created_at first, so a user_id-only filter can't use it efficiently. Use `(user_id, created_at)`.

P5. Orders (100 M rows) page loads need user name and city from 2 other tables, 5,000 reads/s. What helps?

Answer: Denormalise name/city into an order read model or materialized view (or cache) kept in sync via events/CDC; keep normalised tables as source of truth.

P6. Replica lag spikes to 30 s during a nightly backfill. What went wrong and what is the fix?

Answer: Huge single-threaded replay of large writes on replicas. Backfill in small throttled batches, off-peak; monitor lag and pause when it grows.

Interview Q&A (30-second answers)

Q1. Explain ACID.

Atomicity (all or nothing), Consistency (valid state to valid state), Isolation (concurrent = as if serial, via locks/MVCC at chosen level), Durability (committed data survives crashes via WAL + fsync).

Q2. Isolation levels and the anomalies they allow?

Read committed (no dirty reads), repeatable read (stable snapshot), serializable (no anomalies, may abort). Weaker levels allow non-repeatable reads, phantoms, lost updates and write skew.

Q3. How do indexes work and what do they cost?

B-tree gives O(log n) lookups and sorted access; they speed reads but slow writes, use memory/disk; column order (leftmost prefix) and selectivity matter.

Q4. Leader–follower vs multi-leader replication?

Single writer + read replicas (simple, read scaling, lag, failover promotion) vs multiple writers (write HA/geo-latency, but conflicts and weaker consistency).

Q5. What problems does replication solve and not solve?

Solves read scaling and availability; doesn't scale writes or data size, introduces lag, and isn't a backup.

Q6. Federation vs sharding?

Federation splits by function/table group into separate DBs; sharding splits rows of one table across DBs by a shard key. Federation is simpler but limited by the biggest function.

Q7. How do you choose a shard key and handle resharding?

High cardinality, even, matches dominant queries, stable. Use consistent hashing or many logical shards to move data in small units; avoid mod N.

Q8. When would you denormalise?

Read-dominated workloads where joins are costly or data is distributed; keep a normalised source of truth and sync via materialized views/CDC.

Q9. How do you change a schema without downtime?

Expand/contract: add nullable, dual-write, backfill in batches, switch reads, then drop; every step compatible with both app versions; online DDL tools.

Q10. When is a relational database the wrong choice?

Very high write throughput/volume with simple access patterns, flexible schemas, massive scale-out needs or graph/time-series/wide-column workloads — consider NoSQL/NewSQL (Chapter 10).

AMJESH
https://singhamjesh.github.io/

Chapter 9 Cheat Sheet

⚡ **QUICK RECALL** — **Revise this in 2 minutes**

- **RDBMS** = tables + keys + joins + schema + SQL. Normalise for correctness; denormalise selectively for reads.
- **ACID:** Atomic (all/nothing, WAL) • Consistent (valid state) • Isolated (locks/MVCC; read committed → repeatable read → serializable) • Durable (WAL + fsync).
- **Concurrency:** atomic UPDATE first; SELECT FOR UPDATE (pessimistic); version column (optimistic); retry deadlocks; keep transactions short.
- **Index = B-tree**, O(log n); leftmost-prefix; covering; costs space and write speed; verify with EXPLAIN. No functions on indexed columns.
- **Tuning order:** measure (slow log, EXPLAIN) → indexes/queries/schema → pool → cache → replicas. Avoid N+1, SELECT *, deep OFFSET.
- **Replication:** leader–follower (reads scale, lag, promotion logic); multi-leader (write HA, conflicts). Replicas ≠ backups. Not a write-scaling tool.
- **Federation** = split by function (forums/users/products): less traffic per DB, parallel writes; cross-DB joins hard. **Sharding** = split rows by key: scales writes + size; pick key carefully, avoid mod N, expect cross-shard pain.
- **Denormalization** = redundant data to avoid joins (materialized views, CDC). Reads 100:1–1000:1 justify it; writes get costlier.
- **Connections** = instances × pool; use PgBouncer; size with Little's law.
- **Operate:** PITR backups + tested restores; Multi-AZ failover; expand/contract migrations compatible with both app versions.
- **Ladder:** tune → vertical → cache → replicas → partition → federate → shard → NewSQL/NoSQL. Don't shard on day one.

AMJESH https://singhamjesh.github.io/